

Here is the complete list of all **24 backend API endpoints** the frontend calls, with exact request/response contracts:

Authentication — /auth

1. POST /auth/login

```
// Request
{ "email": "user@example.com", "password": "secret" }

// Response
{
  "token": "jwt_access_token",
  "profile": {
    "email": "user@example.com",
    "full_name": "Dr. Jane Smith",
    "role": "faculty",
    "appraisal_role": "faculty",
    "department": "CSE",
    "school": "SOET",
    "employee_id": "FAC001",
    "designation": "Assistant Professor",
    "phone": "9999999999",
    "profile_picture_url": null
  }
}
```

2. POST /auth/register

```
// Request
{
  "email": "user@example.com",
  "password": "secret",
  "full_name": "Dr. Jane Smith",
  "role": "faculty",
  "appraisal_role": "faculty",
  "department": "CSE",
  "school": "SOET",
  "employee_id": "FAC001",
  "designation": "Assistant Professor",
  "phone": "9999999999"
}
```

```
// Response – the created profile object (same shape as login profile)
```

3. GET /auth/me

```
// No body – reads JWT from Authorization header
```

```
// Response – same profile shape as login
```

4. PUT /auth/me

```
// Request – any updatable profile fields
```

```
{  
  "full_name": "Dr. Jane Smith",  
  "department": "CSE",  
  "school": "SOET",  
  "designation": "Associate Professor",  
  "phone": "9999999999",  
  "profile_picture_url": "https://..."  
}
```

```
// Response – updated profile object
```

5. POST /auth/change-password

```
// Request
```

```
{ "current_password": "old", "new_password": "new" }
```

```
// Response – { "message": "Password changed successfully" } or error
```

6. POST /auth/forgot-password

```
// Request
```

```
{ "email": "user@example.com" }
```

```
// Response – { "message": "Reset link sent" }
```

7. POST /auth/reset-password

```
// Request
{ "token": "reset_token_from_email_link", "new_password": "newsecret" }

// Response – { "message": "Password reset successful" }
```

Teaching Appraisal — /appraisal

8. GET /appraisal/snapshot

```
// Query params: ?academic_year=2024-25
// Auth: JWT (reads email from token – faculty sees only their own)

// Response
{
  "payload": {
    "form": {
      "info": { "name": "...", "department": "...", ... },
      "lectures": [...],
      "courseFile": [...],
      "innovDetails": { ... },
      "innovScore": { ... },
      "projects": [...],
      "quals": [...],
      "feedback": { ... },
      "deptActs": [...],
      "uniActs": [...],
      "society": { ... },
      "industry": { ... },
      "acr": { ... },
      "journals": [...],
      "books": [...],
      "ict": { ... },
      "research": { ... },
      "projects2": [...],
      "externalProjects": [...],
      "patents": [...],
      "awards": [...],
      "confs": [...],
      "proposals": [...],
      "products": [...],
      "fdps": [...],

```

```

        "training": [...],
        "sectionSaveStatus": { "lectures": true, ... }
    },
    "totals": { "partATotal": 45.5, "partBTTotal": 30.0, "grandTotal": 75.5 },
    "docs": { "journals-0": [{ "name": "...", "type": "...", "url": "...", "publicId":
"..."}] },
    "submitterProfile": { ... },
    "savedAt": "2024-12-01T10:00:00Z"
}
}
// Return null/404 if no snapshot exists yet

```

9. PUT /appraisal/snapshot

```

// Upsert draft – identified by JWT email + academic_year
// Request
{
  "academic_year": "2024-25",
  "payload": {
    "form": { ...all form sections... },
    "totals": { "partATotal": 45.5, "partBTTotal": 30.0, "grandTotal": 75.5 },
    "docs": { ... },
    "submitterProfile": { ... },
    "savedAt": "2024-12-01T10:00:00Z"
  }
}

// Response – { "message": "Saved" } or the updated snapshot row

```

10. POST /appraisal/submit

```

// Finalizes the appraisal – sets status to submitted, creates declaration row
// Request
{
  "academic_year": "2024-25",
  "form": { ...all form sections... },
  "totals": { "partATotal": 45.5, "partBTTotal": 30.0, "grandTotal": 75.5 },
  "docs": { ... },
  "submitter_profile": { "email": "...", "full_name": "...", ... }
}

// Response – { "message": "Submitted successfully", "submitted_at": "2024-12-
01T10:05:00Z" }

```

11. GET /appraisal/status

```
// Query params: ?academic_year=2024-25
// Auth: JWT – returns status for the logged-in faculty

// Response
{
  "declaration": {
    "faculty_email": "user@example.com",
    "academic_year": "2024-25",
    "status": "pending_hod",
    "submitted_at": "2024-12-01T10:05:00Z",
    "part_a_total": 45.5,
    "part_b_total": 30.0,
    "grand_total": 75.5
  },
  "reviews": [
    {
      "reviewer_role": "hod",
      "reviewer_email": "hod@example.com",
      "status": "reviewed",
      "part_a_score": 44.0,
      "part_b_score": 29.0,
      "total_score": 73.0,
      "remarks": "Good performance",
      "reviewed_at": "2024-12-05T09:00:00Z"
    }
  ]
}
// Return { declaration: null, reviews: [] } if not yet submitted
```

Documents — /appraisal-documents

12. GET /appraisal-documents

```
// Query params: ?academic_year=2024-25
// Auth: JWT – returns documents for logged-in faculty

// Response
[
  {
    "id": 1,
    "faculty_email": "user@example.com",
```

```
"academic_year": "2024-25",
"doc_key": "journals-0",
"section": "journals",
"row_no": 1,
"file_name": "paper.pdf",
"file_type": "application/pdf",
"file_url": "https://storage.example.com/...",
"storage_path": "faculty/user/journals-0.pdf"
}
]
```

Dashboard / Review Queue — /dashboard

13. GET /dashboard/subordinates

```
// For reviewers (HOD, director, dean, VC) to see their queue
// Auth: JWT – backend filters by the logged-in reviewer's role + school/department
// Query params: ?academic_year=2024-25&schools=SOET,SOMC (schools is comma-separated)
```

```
// Response
```

```
[
  {
    "email": "faculty@example.com",
    "name": "Dr. Jane Smith",
    "department": "CSE",
    "school": "SOET",
    "appraisalRole": "faculty",
    "status": "pending_hod",
    "submittedOn": "2024-12-01",
    "selfPartA": 45.5,
    "selfPartB": 30.0,
    "selfTotal": 75.5,
    "hodPartA": 0,
    "hodPartB": 0,
    "hodTotal": 0,
    "hodRemarks": "",
    "directorPartA": 0,
    "directorPartB": 0,
    "directorTotal": 0,
    "directorRemarks": "",
    "deanPartA": 0,
    "deanPartB": 0,
    "deanTotal": 0,
    "deanRemarks": ""
  }
]
```

```
    "vcPartA": 0,  
    "vcPartB": 0,  
    "vcTotal": 0,  
    "vcRemarks": ""  
  }  
]
```

14. GET /dashboard/faculty/{email}

```
// Load a specific faculty member's full snapshot for the reviewer to inspect  
// Auth: JWT – reviewer must be authorized for this faculty  
// Path param: email (URL-encoded)  
// Query params: ?academic_year=2024-25  
  
// Response – the full snapshot payload for that faculty  
// Same shape as GET /appraisal/snapshot response (the payload object)  
// Plus declaration/review summary fields merged in
```

Appraisal Remarks — /appraisal-remarks

All five remark endpoints share the same request/response shape. The {email} path param is the **faculty/subject email** (URL-encoded).

15. PUT /appraisal-remarks/hod/{email}

16. PUT /appraisal-remarks/center-head/{email}

17. PUT /appraisal-remarks/director/{email}

18. PUT /appraisal-remarks/dean/{email}

19. PUT /appraisal-remarks/final/{email} ← this is the VC endpoint

```
// Request (same for all five)  
{  
  "academic_year": "2024-25",  
  "remarks": "Excellent performance in research.",  
  "part_a_score": 44.0,
```

```

    "part_b_score": 29.0,
    "total_score": 73.0,
    "section_scores": {
      "lectures": 18.0,
      "research": 12.0
    }
  }
}

// Response
{
  "message": "Review submitted",
  "status": "pending_director" // updated appraisal status after this review
}

```

The status progression is:

- After HOD → pending_director
- After center_head → pending_director
- After director → pending_dean
- After dean → pending_vc
- After VC (final) → completed

Non-Teaching Appraisal — /non-teaching

20. GET /non-teaching/appraisal

```

// Query params: ?academic_year=2024-25
// Auth: JWT – returns for logged-in non-teaching staff

// Response
{
  "staff_email": "staff@example.com",
  "academic_year": "2024-25",
  "status": "pending_ro",
  "submitted_at": "2024-12-01T10:00:00Z",
  "payload": {
    "status": "pending_ro",
    "submittedByRole": "non_teaching_staff",
    "info": { "name": "...", "employeeId": "...", "designation": "...", ... },
    "partA": { ... },
    "partB": { ... }
  },
  "self_total": 72.0,
  "ro_total": null,
  "registrar_total": null,

```



```
"vc_total": null
}
// Return null/404 if not yet submitted
```

21. PUT /non-teaching/appraisal

```
// Create or update self-appraisal (also used for re-submission)
// Request
{
  "staff_email": "staff@example.com",
  "academic_year": "2024-25",
  "payload": {
    "status": "pending_ro",
    "submittedByRole": "non_teaching_staff",
    "info": { ... },
    "partA": { ... },
    "partB": { ... }
  },
  "status": "pending_ro"
}

// Response – same as GET /non-teaching/appraisal response shape
```

22. GET /non-teaching/subordinates

```
// For reporting_officer, registrar, vc to see their review queue
// Auth: JWT – backend filters by logged-in reviewer's role
// Query params: ?academic_year=2024-25

// Response
[
  {
    "staff_email": "staff@example.com",
    "name": "Mr. John Doe",
    "designation": "Lab Assistant",
    "department": "CSE",
    "appraisalRole": "non_teaching_staff",
    "status": "pending_ro",
    "submittedOn": "2024-12-01",
    "selfTotal": 72.0,
    "roTotal": 0,
    "registrarTotal": 0,
    "vcTotal": 0,
    "payload": { ... }
  }
]
```

```
]
```

23. PUT /non-teaching/review/{email}

```
// Path param: staff email (URL-encoded)
// Request
{
  "academic_year": "2024-25",
  "payload": {
    "status": "pending_registrar",
    "roRemarks": "Good work",
    "partA": { ... },
    "partB": { ... }
  },
  "status": "pending_registrar",
  "remarks": "Good work"
}

// Response – the updated non-teaching appraisal row
// Same shape as GET /non-teaching/appraisal
```

Non-teaching status progression:

- Non-teaching staff submits → pending_ro
- After reporting_officer → pending_registrar
- After registrar → pending_vc
- After VC → completed

File Upload — /upload

24. POST /upload

```
// Request: multipart/form-data
// Field name: "file" (single file per request)
// Content-Type: multipart/form-data (set automatically by browser/axios)

// Response
{
  "url": "https://storage.example.com/faculty/uploads/file.pdf",
  "publicId": "faculty/uploads/file",
  "name": "file.pdf",
  "type": "application/pdf"
}
```

```
}
```

The frontend stores { url, publicId, name, type } in the docs state map. The url is what gets stored as file_url in appraisal_documents.

Summary Table

#	Method	Path	Who calls it
1	POST	/auth/login	Login page
2	POST	/auth/register	Signup page
3	GET	/auth/me	EditProfile load
4	PUT	/auth/me	EditProfile save
5	POST	/auth/change-password	EditProfile
6	POST	/auth/forgot-password	Login page
7	POST	/auth/reset-password	ResetPassword page
8	GET	/appraisal/snapshot	Dashboard (load draft)
9	PUT	/appraisal/snapshot	Dashboard (auto-save)
10	POST	/appraisal/submit	Dashboard (final submit)
11	GET	/appraisal/status	All faculty dashboards
12	GET	/appraisal-documents	Dashboard (load docs)
13	GET	/dashboard/subordinates	HOD/Director/Dean/VC dashboards
14	GET	/dashboard/faculty/{email}	Reviewer viewing individual faculty
15	PUT	/appraisal-remarks/hod/{email}	HOD dashboard
16	PUT	/appraisal-remarks/center-head/{email}	Center head dashboard
17	PUT	/appraisal-remarks/director/{email}	Director dashboard
18	PUT	/appraisal-remarks/dean/{email}	Dean dashboards
19	PUT	/appraisal-remarks/final/{email}	VC dashboard
20	GET	/non-teaching/appraisal	NonTeachingStaffDashboard
21	PUT	/non-teaching/appraisal	NonTeachingStaffDashboard
22	GET	/non-teaching/subordinates	RO/Registrar/VC dashboards
23	PUT	/non-teaching/review/{email}	RO/Registrar/VC dashboards
24	POST	/upload	All dashboards (file attach)

All endpoints except /auth/login, /auth/register, /auth/forgot-password, and /auth/reset-password require the Authorization: Bearer <token> header. The backend should extract the faculty/reviewer email from the JWT on every authenticated request.

